



PRÉSENTATION TECHNIQUE

Les fonctions en PHP

Organiser, réutiliser et structurer son code

Guide simple et détaillé — Sonatel Academy
Préparé pour Khadija — Dakar, Sénégal

1. Qu'est-ce qu'une fonction ?

DÉFINITION

Une fonction est un **bloc de code réutilisable**, auquel on donne un nom, qui réalise une tâche précise, et qu'on peut appeler autant de fois qu'on veut sans avoir à le réécrire à chaque fois.

ANALOGIE SIMPLE

Une fonction, c'est comme une recette de cuisine. Tu écris la recette une seule fois ("préparer un thé"), et ensuite tu peux la réutiliser autant de fois que tu veux, avec des ingrédients différents à chaque fois (eau chaude, feuilles de thé différentes), sans jamais réécrire toute la recette.

Pourquoi utiliser des fonctions ?

- **Éviter de répéter du code** : au lieu de copier-coller le même bloc partout, on l'écrit une fois dans une fonction.
- **Rendre le code plus lisible** : un nom de fonction explique ce que fait le bloc de code, sans avoir à le lire en détail.
- **Faciliter la correction** : si une erreur existe, on la corrige à un seul endroit au lieu de la chercher partout.
- **Organiser un gros projet** : chaque fonction s'occupe d'une seule tâche précise.

Créer et appeler une fonction simple

Pour créer une fonction, on utilise le mot-clé `function`, suivi d'un nom et de parenthèses.

```
<?php
function direBonjour() {
    echo "Bonjour tout le monde !";
}

direBonjour(); // Appel de la fonction → Affiche : Bonjour tout le monde !
direBonjour(); // On peut l'appeler autant de fois qu'on veut
?>
```

ASTUCE PÉDAGOGIQUE

Pour expliquer à tes camarades : écrire `function direBonjour() { ... }` ne fait qu'enregistrer la recette, ça **n'exécute rien**. Il faut "appeler" la fonction avec `direBonjour();` pour qu'elle s'exécute réellement.

2. Les paramètres : rendre une fonction flexible

DÉFINITION

Un **paramètre** est une information qu'on transmet à une fonction au moment de l'appeler, pour qu'elle puisse l'utiliser et adapter son comportement. Cela évite de créer une fonction différente pour chaque cas.

Une fonction avec un paramètre

```
<?php
function saluer($nom) {
    echo "Bonjour " . $nom . " !";
}

saluer("Khadija"); // Affiche : Bonjour Khadija !
saluer("Moussa"); // Affiche : Bonjour Moussa !
?>
```

ANALOGIE SIMPLE

Reprenons la recette de thé : le paramètre, c'est l'ingrédient que tu changes à chaque fois (thé vert, thé noir...). La recette (la fonction) reste la même, mais le résultat change selon ce que tu lui donnes.

Une fonction avec plusieurs paramètres

On peut donner autant de paramètres que nécessaire, séparés par des virgules.

```
<?php
function presenter($nom, $ville, $age) {
    echo $nom . " a " . $age . " ans et vit à " . $ville;
}

presenter("Khadija", "Dakar", 19);
// Affiche : Khadija a 19 ans et vit à Dakar
?>
```

PIÈGE FRÉQUENT

L'ordre des arguments donnés lors de l'appel doit correspondre à l'ordre des paramètres définis dans la fonction. Si on inverse `presenter("Dakar", "Khadija", 19)`, le résultat sera incorrect car PHP associe les valeurs dans l'ordre où elles sont écrites.

Les valeurs par défaut

On peut donner une valeur par défaut à un paramètre. Si l'appel ne fournit pas cette valeur, PHP utilisera celle par défaut.

```
<?php
function saluer($nom = "invité") {
    echo "Bonjour " . $nom;
}

saluer("Khadija"); // Affiche : Bonjour Khadija
saluer();          // Affiche : Bonjour invité
?>
```

3. Le mot-clé return : renvoyer un résultat

DÉFINITION

`return` permet à une fonction de **renvoyer une valeur** à l'endroit où elle a été appelée, pour que cette valeur puisse être utilisée ensuite (stockée dans une variable, affichée, réutilisée dans un calcul...).

Différence entre echo et return

C'est l'une des confusions les plus fréquentes chez les débutants : `echo` affiche directement à l'écran, alors que `return` renvoie une valeur que l'on peut ensuite réutiliser ailleurs dans le programme.

```
<?php
function additionner($a, $b) {
    return $a + $b;
}

$resultat = additionner(5, 3); // $resultat vaut 8
echo "Le total est : " . $resultat; // Affiche : Le total est : 8
?>
```

ASTUCE PÉDAGOGIQUE

Pour bien faire comprendre à tes camarades : dès que PHP rencontre `return`, la fonction **s'arrête immédiatement** et renvoie la valeur. Tout code écrit après un `return` dans la fonction ne sera jamais exécuté.

echo	return
Affiche une valeur à l'écran	Renvoie une valeur utilisable ailleurs
Ne peut pas être stocké dans une variable	Peut être stocké dans une variable
N'arrête pas la fonction	Arrête immédiatement la fonction
Utilisé pour montrer un résultat final	Utilisé pour transmettre un résultat à réutiliser

Exemple combiné : calculer puis utiliser le résultat

```
<?php
function estMajeur($age) {
    if ($age >= 18) {
        return true;
    }
    return false;
}

if (estMajeur(20)) {
    echo "Accès autorisé";
} else {
    echo "Accès refusé";
}

?>
// Résultat : Accès autorisé
```

4. La portée des variables et les fonctions PHP intégrées

La portée des variables dans une fonction (rappel important)

Une variable créée à l'intérieur d'une fonction n'existe que dans cette fonction : elle est isolée du reste du programme.

```
<?php
function calculer() {
    $total = 100;    // existe seulement à l'intérieur de calculer()
    return $total;
}

calculer();
echo $total;    // Erreur : $total n'existe pas en dehors de la fonction
?>
```

PIÈGE FRÉQUENT

Une variable définie en dehors d'une fonction n'est pas non plus automatiquement accessible à l'intérieur de cette fonction. PHP isole volontairement chaque fonction pour éviter les conflits entre variables portant le même nom.

Les fonctions déjà intégrées dans PHP

En plus des fonctions qu'on crée soi-même, PHP fournit énormément de fonctions déjà prêtes à l'emploi, pour gagner du temps.

Fonction	Utilité	Exemple
strlen()	Compte le nombre de caractères	strlen("Dakar") → 5
strtoupper()	Met en majuscules	strtoupper("salut") → SALUT
strtolower()	Met en minuscules	strtolower("SALUT") → salut
count()	Compte les éléments d'un tableau	count(\$fruits)
rand()	Génère un nombre aléatoire	rand(1, 10)
gettype()	Donne le type d'une variable	gettype(\$age)

```
<?php
$prenom = "khadija";
echo strtoupper($prenom);    // Affiche : KHADIJA
echo strlen($prenom);        // Affiche : 7
?>
```

ASTUCE PÉDAGOGIQUE

Tu peux expliquer à tes camarades que ces fonctions existent pour ne pas réinventer la roue : pas besoin d'écrire soi-même le code pour compter des lettres ou mettre un texte en majuscules, PHP le fait déjà.

5. Résumé pour bien expliquer les fonctions

Concept	Exemple de code
Créer une fonction	<code>function direBonjour() { ... }</code>
Appeler une fonction	<code>direBonjour();</code>
Fonction avec paramètre	<code>function saluer(\$nom) { ... }</code>
Valeur par défaut	<code>function saluer(\$nom = "invité")</code>
Renvoyer un résultat	<code>return \$a + \$b;</code>
Récupérer le résultat	<code>\$x = additionner(5, 3);</code>
Fonction PHP intégrée	<code>strtoupper(\$texte);</code>

La phrase à retenir pour ta présentation

"Une fonction est un bloc de code réutilisable qu'on définit une seule fois avec le mot-clé `function`, et qu'on peut appeler autant de fois qu'on veut, avec des paramètres différents à chaque fois. Le mot-clé `return` permet à la fonction de renvoyer un résultat qu'on pourra réutiliser ailleurs dans le programme."

Erreurs courantes à éviter

- Oublier d'appeler la fonction après l'avoir définie (définir ≠ exécuter).
- Confondre `echo` (afficher) et `return` (renvoyer une valeur réutilisable).
- Donner les arguments dans le mauvais ordre lors de l'appel d'une fonction.
- Croire qu'une variable créée dans une fonction est accessible en dehors.
- Écrire du code après un `return` en pensant qu'il sera exécuté (il ne le sera jamais).